

TradingAgents

Deployment Guide for Self-Hosted Coolify v4

Version 1.1 | Ubuntu VM | Coolify at **`http://192.168.0.161:8000`**

This guide walks you through deploying the TradingAgents multi-agent LLM financial trading framework on your self-hosted Coolify instance. It is written for the Coolify dashboard fields shown in your screenshot and covers the exact build-pack, port, and environment settings needed to deploy without errors.

1. Why the default deployment fails

In the Coolify dashboard screenshot the resource is configured as a **Nixpacks** application with **Ports Expose = 3000** and a long set of Nix-specific **Custom Docker Options**. That is the wrong build pack for this project and it produces a build/deploy error.

- **TradingAgents is not a Nixpacks / static-site project.** It is a Python project that ships its own **Dockerfile** and **docker-compose.coolify.yml**.
- **Nixpacks cannot auto-detect the correct start command** because the entry point is a custom **uvicorn** command inside the Dockerfile.
- **Port 3000 is wrong** for this app. The container listens internally on **8000**; Coolify itself already uses host port **8000**, so the app must be exposed on a different host port (e.g. **8001**) or through a Coolify domain.
- **The Nixpacks custom Docker options must be removed.** They are only needed when Coolify builds with Nixpacks.

Fix: change the **Build Pack** to **Docker Compose** (recommended) or **Dockerfile**, clear the custom options, and expose the correct port.

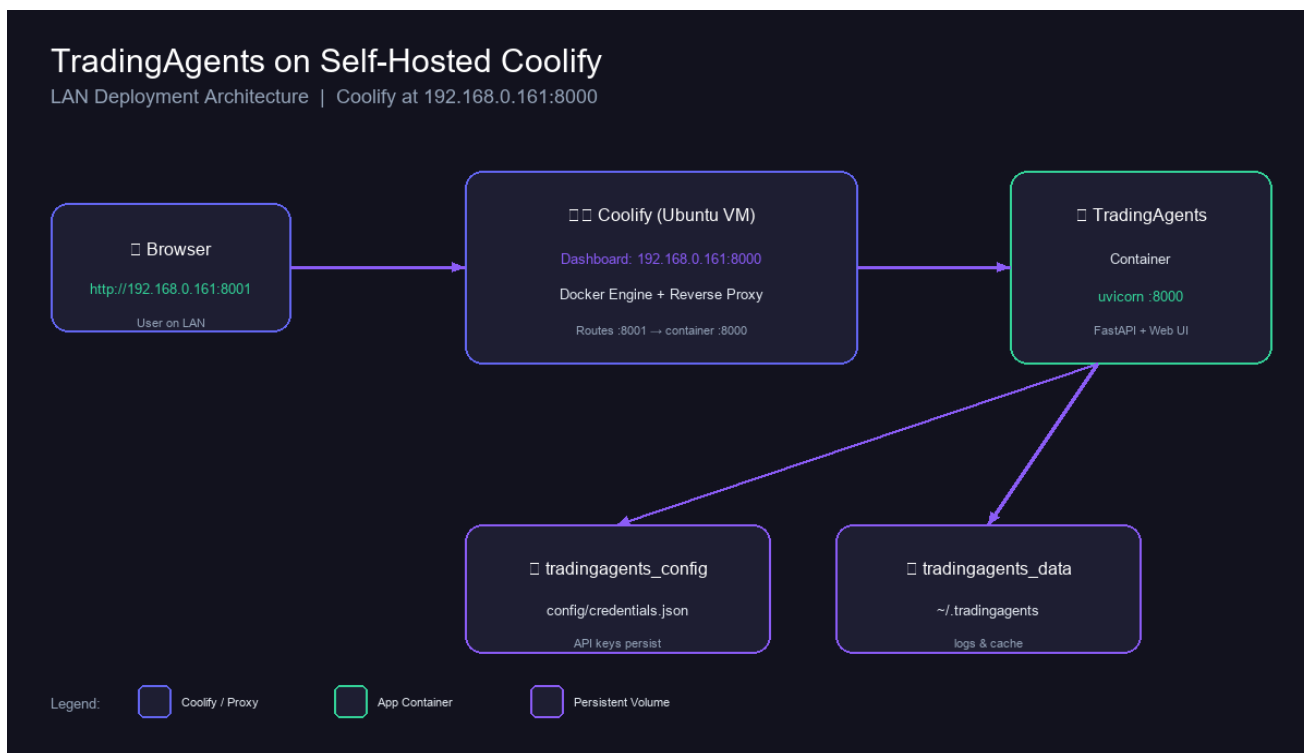
2. Prerequisites

- **Ubuntu VM** with Docker installed and running.
- **Coolify v4** installed and accessible at **http://192.168.0.161:8000**.
- **Git repository** (GitHub, GitLab, or Gitea) containing the TradingAgents project.
- **LLM API key** for at least one provider (OpenAI, Anthropic, Google, etc.) — or enter it later in the browser.
- Network access from your browser to the VM on the LAN.

Note: Coolify itself occupies host port **8000**. The TradingAgents app will be exposed on host port **8001** to avoid a conflict.

3. Architecture Overview

The diagram below shows how the browser, Coolify, the app container, and persistent volumes fit together on your LAN.



Key points: The browser reaches the app at **http://192.168.0.161:8001**. Coolify routes that to the container's internal port **8000**. API keys saved in the browser persist to the **tradingagents_config** volume.

4. Recommended Coolify Configuration

Use the **Docker Compose** build pack. It reads the project's **docker-compose.coolify.yml** file, so ports, environment variables, volumes, and the healthcheck are applied automatically.

4.1 General tab

Field	Value
Build Pack	Docker Compose
Base Directory	/
Docker Compose Location	docker-compose.coolify.yml
Is it a static site?	No / unchecked
Custom Docker Options	Leave empty

Important: Do **not** select **Nixpacks**. The Nixpacks options you saw in the screenshot are not used for this project.

4.2 Network tab

The **docker-compose.coolify.yml** file already maps the correct ports. You only need to make sure Coolify exposes the container port:

```
Ports Expose: 8000
Port Mapping: 8001:8000 # host 8001 -> container 8000
```

If you prefer to use Coolify's automatic domain/SSL instead of an IP:port, set **Ports Expose = 8000** and leave the Port Mapping blank. Coolify will proxy the generated domain to port 8000 inside the container.

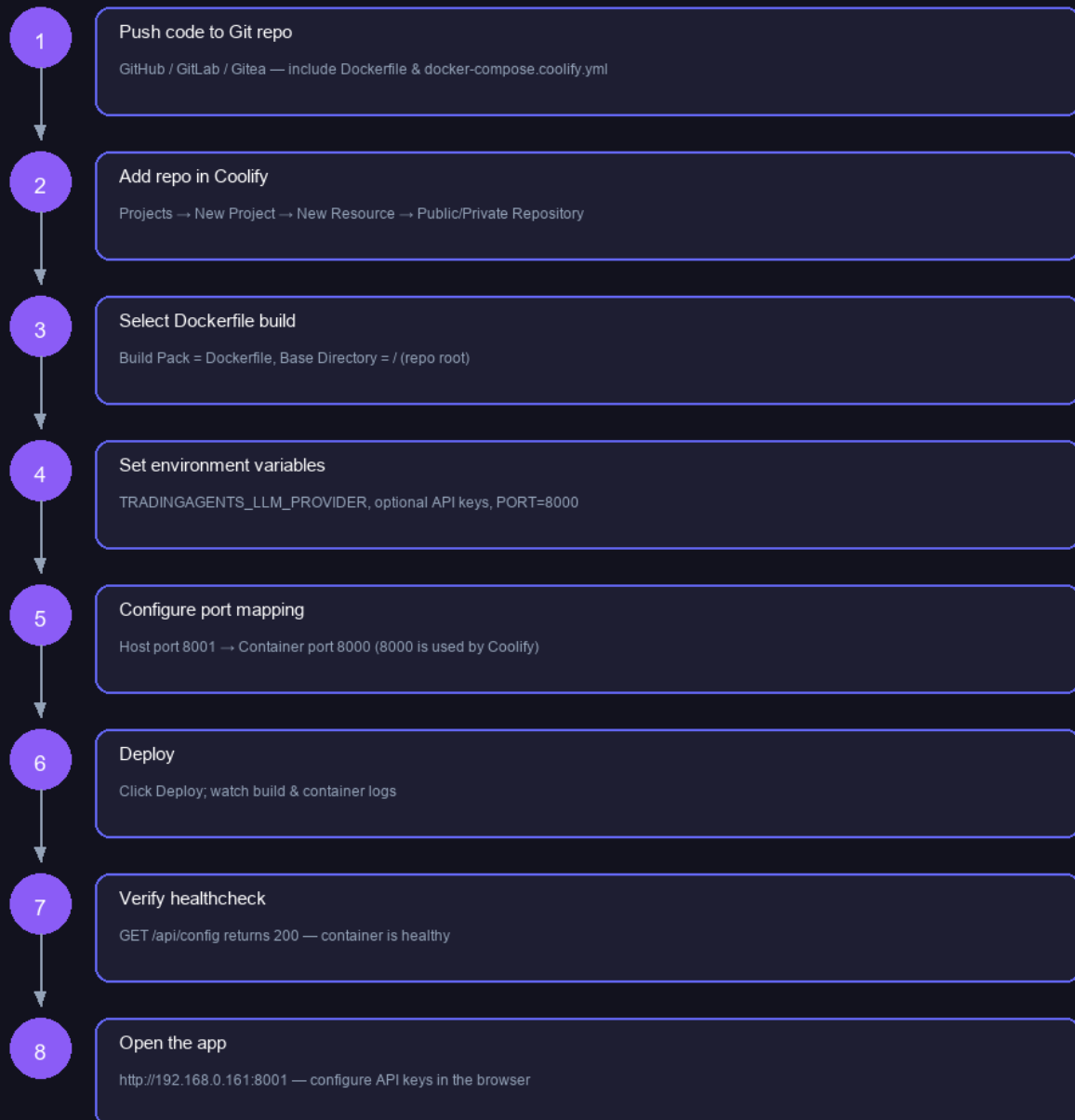
4.3 Environment variables

The Docker Compose file already sets **PORT=8000**. Add only the variables you need. At minimum you can leave all API-key variables empty and enter the key later in the browser.

```
PORT=8000
TRADINGAGENTS_LLM_PROVIDER=openai
# Optional: pre-seed API keys (otherwise save them in the browser)
# OPENAI_API_KEY=sk-...
# ANTHROPIC_API_KEY=sk-ant-...
# GOOGLE_API_KEY=AIZA...
# CLOUDFLARE_ACCOUNT_ID=...
# CLOUDFLARE_GATEWAY_ID=...
# CLOUDFLARE_BYOK_ALIAS=default
```

5. Step-by-Step Deployment

Deployment Flow — Step by Step



5.1 Push the code to a Git repository

- 1 Create a repository on GitHub, GitLab, or Gitea.
- 2 Push the TradingAgents project (including **Dockerfile**, **pyproject.toml**, and **docker-compose.coolify.yml**).
- 3 Ensure **config/credentials.json** is NOT committed (it is in **.gitignore**).

5.2 Add the repository in Coolify

- 1 Log in to Coolify at **http://192.168.0.161:8000**.
- 2 Go to **Projects** → **New Project** → give it a name (e.g. 'TradingAgents').
- 3 Click **New Resource** → **Public Repository** (or Private Repository / GitHub App if needed).
- 4 Paste the repository URL and select the branch (e.g. **main**).

5.3 Configure the build pack

- 1 Open the **General** tab of the new resource.
- 2 Set **Build Pack** to **Docker Compose**.
- 3 Set **Base Directory** to **/**.
- 4 Set **Docker Compose Location** to **docker-compose.coolify.yml**.
- 5 Make sure **Custom Docker Options** is empty.

5.4 Configure the network

- 1 Open the **Network / Ports** section.
- 2 Set **Ports Expose** to **8000**.
- 3 Set **Port Mapping** to **8001:8000** (host 8001 → container 8000).
- 4 Leave the mapping blank if you will use a Coolify-generated domain instead.

5.5 Set environment variables (optional)

- 1 Open the **Environment Variables** tab.
- 2 Add **PORT=8000** if it is not already inherited from the compose file.
- 3 Add **TRADINGAGENTS_LLM_PROVIDER=openai** (or google, anthropic, etc.).
- 4 API keys can be left empty and entered later in the browser.

5.6 Deploy

- 1 Click **Deploy**.
- 2 Watch the build logs — Coolify will build the Docker image and start the container.
- 3 Wait for the status to show **Running** and the healthcheck to pass.

5.7 Verify the healthcheck

The compose file's healthcheck calls **GET /api/config** on port 8000. A healthy container returns HTTP 200. You can verify manually:

```
curl -f http://192.168.0.161:8001/api/config
```

5.8 Open the app

Open **http://192.168.0.161:8001** in your browser. You should see the TradingAgents configuration page.

6. Dockerfile Build Pack (alternative)

If you prefer not to use Docker Compose, you can deploy with the **Dockerfile** build pack instead. In that case you must configure ports, environment variables, and storage manually in the Coolify UI.

- **Build Pack** = Dockerfile
- **Base Directory** = /
- **Ports Expose** = 8000
- **Port Mapping** = 8001:8000
- Add the environment variables from section 4.3 manually.
- Add two **Persistent Storage** mounts: **/home/appuser/.tradingagents** and **/home/appuser/app/config**.

7. Configure API Keys in the Browser

TradingAgents lets you save API keys directly in the browser — no need to set them in Coolify. Keys persist to **config/credentials.json** inside the container, backed by the **tradingagents_config** volume.

- 1 Open **http://192.168.0.161:8001** (or your Coolify domain).
- 2 In the **LLM Configuration** section, select your provider (e.g. OpenAI).
- 3 Enter your API key in the **API Key** field.
- 4 Click **Save Key** — a checkmark confirms it was saved to the project config.
- 5 Click **Validate & Fetch Models** to test the key and load available models.
- 6 Select your Deep & Quick models, then click **Launch Analysis**.

Saved keys are loaded into the environment at startup, so analysis works immediately. Keys are stored in plaintext (like .env) and are excluded from version control via .gitignore.

8. Persistent Storage

Two Docker volumes persist data across container restarts and redeploys:

Volume	Mount point	Purpose
--------	-------------	---------

tradingagents_config	/home/appuser/app/config	API keys & settings (credentials.json)
tradingagents_data	/home/appuser/.tradingagents	Logs, cache, memory

9. Troubleshooting

9.1 Build fails or container exits immediately

- Make sure the **Build Pack** is **Docker Compose** (or Dockerfile), not Nixpacks.
- Confirm **Ports Expose** is **8000**, not 3000.
- Check that **Custom Docker Options** does not contain Nixpacks flags.
- Look at the **Logs** tab in Coolify for the exact error.

9.2 Healthcheck failing

If the container shows unhealthy, check that the app is listening on port 8000 inside the container and that the Dockerfile CMD uses uvicorn:

```
CMD ["uvicorn", "app.server:create_app", "--factory", "--host", "0.0.0.0",
"--port", "8000"]
```

9.3 Port conflict

Coolify uses host port 8000. If you try to map the app to host port 8000 as well, the container will fail to start. Use host port 8001 (or another free port) and point Coolify at container port 8000.

9.4 Cannot reach the app from browser

- Verify the VM firewall allows inbound traffic on port 8001.
- Confirm the container is running: **docker ps** on the VM.
- Check the app logs in Coolify for startup errors.
- If you used a Coolify domain, make sure DNS resolves to the VM.

9.5 API key not working

- Ensure you clicked **Save Key** (not just typed the key).
- Verify the key is valid with **Validate & Fetch Models**.
- Check that the **tradingagents_config** volume is mounted so the key persists.

10. Security Notes

- **config/credentials.json** is excluded from git via .gitignore — never commit API keys.
- Keys are stored in plaintext (same as .env). Protect the VM and volume access.
- The app is exposed on the LAN. Restrict access with a firewall or Coolify's authentication if needed.
- Use Coolify's environment variables for non-secret config; use the browser Save Key feature for secrets.

11. Quick Reference

- Coolify dashboard: **http://192.168.0.161:8000**
- TradingAgents app: **http://192.168.0.161:8001**
- Build pack: **Docker Compose** with **docker-compose.coolify.yml**
- Container port: **8000**
- Host port: **8001**
- Healthcheck: **GET /api/config**
- Config store: **config/credentials.json** (volume: tradingagents_config)
- Web server: **Dockerfile** → uvicorn on 0.0.0.0:8000